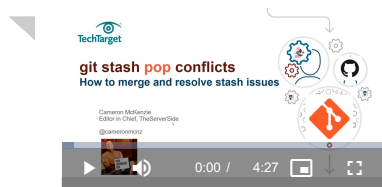


How to easily merge and resolve git stash pop conflicts

Cameron McKenzie, TechTarget

Published: 20 Jul 2020



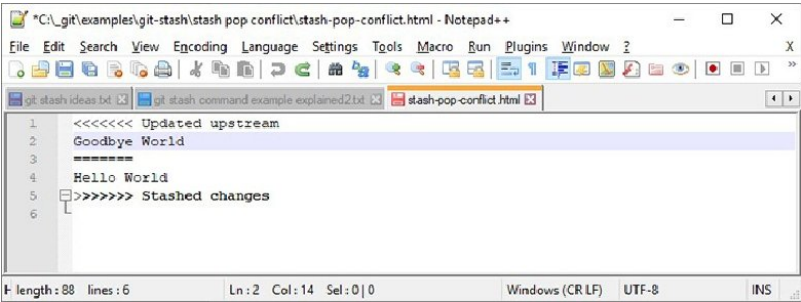
There's always the chance that a merge conflict might occur when a developer updates or overwrites a file. Here's how to merge and resolve `git stash pop` conflicts when they arise, and clean up the stash history so it appears that the `git stash` conflict never actually happened.

When the `pop` command runs, it's expected that files from the stash will overwrite the contents of the files in the local [working tree](#), and the updated files will be staged in the git index. But if a `git stash pop` conflict arises, then the problematic files won't be added to the index. Instead, Git populates the conflicted files with content from both the local working tree and the stash.

Here is what a developer will see in the terminal window of a `git stash pop` conflict:

```
/examples/git-stash/stash pop conflict (master)
$ git stash pop
Auto-merging stash-pop-conflict.html
CONFLICT (content): Merge conflict in stash-pop-conflict.html
The stash entry is kept in case you need it again.
```

There's no magic remedy for such merge conflicts. The only option for developers is to edit the file by hand and keep what they want and dispose of what they don't want. Once they merge and save the file, they will have effectively resolved the `git stash` conflict.



 When a git stash pop conflict occurs, the developer must manually merge and resolve problems.

Clean up a git stash pop conflict

Unfortunately, when the `git stash pop` conflict resolves, the merged file isn't staged. A developer will need to do this manually with the `git add` command. Once staged, the updated file will be included as part of the next commit.

A noteworthy byproduct of a `git stash pop` conflict is the fact that the restored entry from the stash history isn't deleted. The `pop` command is a combination of `git stash apply` and `git stash drop`, but a conflict short-circuits the process and the `drop` command never runs. A developer must additionally run the `git stash drop` command to restore the state of the development environment to where it would be if the `git stash pop` conflict never happened.

Here is an example of a `git stash merge` conflict and the steps used to resolve it.

First, initialize a [Git repository](#), create a file and add it to the index. A [commit with the message](#) "First commit" occurs.

Up Next

Learn how to save to local storage
git stash tutorial

Sometimes developers need to jump off their distributed development branch to local storage. This git stash tutorial to learn the ins and outs of this command.

```
/examples/git-stash/stash pop conflict
$ git init
$ touch stash-pop-conflict.html
$ git add .
$ git commit -m "First commit"
```

The file will then be edited by a developer, stashed, edited again and then made part of a commit. At the end of this following sequence of steps, the file in the stash is out of sync with the committed file in the workspace. This will cause a `git stash pop` conflict.

```
/examples/git-stash/stash pop conflict
$ echo "Hello World" >> stash-pop-conflict.html
$ git stash
$ echo "Goodbye World" >> stash-pop-conflict.html
$ git commit -a -m "Work Done!"
```

Sure enough, the `git stash pop` command causes a problem.

```
/examples/git-stash/stash pop conflict
$ git stash pop
Auto-merging stash-pop-conflict.html
CONFLICT (content): Merge conflict in stash-pop-conflict.html
The stash entry is kept in case you need it again.
```

A developer will need to manually update the file -- in this example, we used the `echo` command to perform the update -- to resolve the issue. After the `git stash merge` conflict is resolved, add the file to the index and commit the change.

```
/examples/git-stash/stash pop conflict
$ echo "This will resolve conflicts." >> stash-pop-conflict.html
$ git add .
$ git commit -m "all merged"
[master 06d8404] all merged
1 file changed, 1 insertion(+), 1 deletion(-)
```

The `git stash pop` command is supposed to delete the latest record in the stash history, but the `git stash list` command shows that this hasn't happened, which is to be expected with a failed pop. This requires the developer to issue a `git stash drop` command manually.

```
$ git stash list
stash@{0}: WIP on master: ffb2b5a First commit

$ git stash drop
```

A [reflog inspection](#) shows two commit messages and a reset. Many developers are surprised that an applied stash registers as a [hard reset](#) in the reflog, but that's part of the process.

```
/examples/git-stash/stash pop conflict (master)
$ git reflog
06d8404 (HEAD -> master) HEAD@{0}: commit: all merged
718f4b0 HEAD@{1}: commit: Work Done!
ffb2b5a HEAD@{2}: reset: moving to HEAD
ffb2b5a HEAD@{3}: commit (initial): First commit
```

And that's how to resolve a `git stash pop` conflict and [restore the workspace](#) to the state it would have been in if the conflict never happened in the first place.

Up Next

◀ [Learn how to save to local stor
git stash tutorial](#) ▶

Sometimes developers need to jump off th
distributed development branch to local st
this git stash tutorial to learn the ins and o

-ADS BY GOOGLE

[SEARCHAPPARCHITECTURE](#) [SOFTWARE QUALITY](#) [CLOUD COMPUTING](#) [SECURITY](#) [AWS](#)



How to handle typical event-driven architecture failures

Adding events to an architecture can pose plenty of problems. Review some common event-driven architecture failures development ...

The 6 non-negotiable REST architecture constraints

While REST-centric design isn't necessarily hard, there are some non-negotiable rules when it comes to resource provisioning. ...

[About Us](#) [Meet The Editors](#) [Contact Us](#) [Advertisers](#) [Business Partners](#) [Media Kit](#) [Corporate Site](#)

[Contributors](#) [Reprints](#) [Answers](#) [Definitions](#) [E-Products](#) [Events](#) [Features](#)

[Guides](#) [Opinions](#) [Photo Stories](#) [Quizzes](#) [Tips](#) [Tutorials](#) [Videos](#)

All Rights Reserved,
Copyright 2000 - 2021, TechTarget

[Privacy Policy](#)

[Do Not Sell My Personal Info](#)